รายงานงานที่ 7
การพัฒนา ทดสอบ และนำเสนอเว็บแอปพลิเคชัน

ConcertPass — ระบบจองตั๋วคอนเสิร์ตผ่านแอปพลิเคชันบนโทรศัพท์มือถือ
Mobile Concert Ticket Booking System

กลุ่มที่ 8
นายทีฆทัศน์ อนุเดช และ นายปรเมษฐ์ ม่วงจ้อย

เสนอ ผู้ช่วยศาสตราจารย์ ดร.เสถียร จันทร์ปลา
รายวิชา การพัฒนาเว็บแอปพลิเคชัน สาขาวิชาวิทยาการคอมพิวเตอร์
คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏสวนสุนันทา

1. ชื่อและวัตถุประสงค์ของโครงงาน

ชื่อโครงงาน: ระบบจองตั๋วคอนเสิร์ตผ่านแอปพลิเคชันบนโทรศัพท์มือถือ (Mobile Concert Ticket Booking System)

ปัญหา: การเลือกซื้อตั๋วคอนเสิร์ตล่าช้า ซับซ้อน ข้อมูลที่นั่งไม่เป็นเรียลไทม์ ค้นหาได้ยาก และพกพา/ตรวจสอบบัตรเข้างานไม่สะดวก

วัตถุประสงค์: พัฒนาเว็บแอปที่ใช้งานได้จริงบนมือถือ รองรับการค้นหา ดูรายละเอียด เลือกที่นั่ง จอง (ชำระเงินจำลอง) และรับ E-Ticket แบบดิจิทัล โดยใช้ React + Vite และ Firebase

กลุ่มผู้ใช้เป้าหมาย: ผู้เข้าชมคอนเสิร์ตและแฟนเพลง (ผู้จัดงานอยู่นอกขอบเขตการพัฒนา)

2. ขอบเขตและฟังก์ชันที่พัฒนาเสร็จ

รหัส	ฟังก์ชัน	หน้า / Component
F01	ดูรายการคอนเสิร์ตและหมวดหมู่ (19 คอนเสิร์ต 5 หมวด)	HomePage, ConcertListPage, ConcertCard, CategoryCard
F02	ค้นหา (ชื่อ ศิลปิน สถานที่) และกรองตามหมวด	SearchBar, ConcertListPage
F03	ดูรายละเอียดคอนเสิร์ต	ConcertDetailPage
F04	เลือกที่นั่งตามโซน (สูงสุด 6 ที่นั่ง/ครั้ง)	SeatBookingPage, SeatGrid, SeatItem
F05	ยืนยันการจองแบบ atomic ป้องกันจองซ้ำ	createBooking(), BookingSummaryBar
F06	E-Ticket พร้อม QR	ETicketPage, TicketCard, QRCodeDisplay
F07	รายการโปรด	FavoritesPage, FavoriteButton
F08	ประวัติการจอง	ETicketPage
F09	สมัครสมาชิก / เข้าสู่ระบบ / ออกจากระบบ (Firebase Authentication)	LoginPage, AuthContext, Protected route

นอกขอบเขต: ชำระเงินจริง, ระบบหลังบ้านผู้จัดงาน, สแกน QR เข้างาน

3. โครงสร้างข้อมูล Firebase

ใช้ Firebase Realtime Database (นำเข้าจาก `firebase/firebase_data.json`) และ Firebase Authentication แบบ Email/Password

Node	คีย์	ฟิลด์หลัก	สิทธิ์ (Rules)
categories	cat_001–005	name_th, name_en, description	อ่านได้ทุกคน เขียนไม่ได้
concerts	C001–C019	name, artist, category_id, venue, event_date, start/end_time, ticket_price, image_*	อ่านได้ทุกคน เขียนไม่ได้
seats	S001–S950	concert_id, zone, seat_number, price, status, booked_by	อ่านได้ทุกคน; เขียนได้เฉพาะ Available→Booked โดย booked_by = uid ตนเอง และห้ามแก้ราคา
bookings	push id	user_id, concert_id, seat_ids, seat_numbers, total_price, booking_date, payment_status	อ่านเฉพาะของตนเอง (query user_id = uid); สร้างได้เมื่อ user_id = uid
favorites	{uid}_{concertId}	user_id, concert_id, created_at	อ่าน/เขียน/ลบเฉพาะของตนเอง
users	{uid}	first_name, last_name, email, phone, role, status	อ่าน/เขียนเฉพาะ uid ตนเอง

ที่นั่ง 950 รายการ (19 คอนเสิร์ต × 50: VIP 10, Standard 20, Economy 20; ราคา VIP = ticket_price, Standard ≈ 80%, Economy ≈ 50%) ถูกเพิ่มจากข้อมูลงานที่ 5 เพื่อให้เลือกที่นั่งได้ทุกคอนเสิร์ต Rules ทั้งหมดอยู่ใน `firebase/database.rules.json` และมี index บน `seats.concert_id`, `bookings.user_id`, `favorites.user_id`

การป้องกันจองซ้ำ: การจองใช้ `update()` แบบ multi-path ชุดเดียวเขียน `seats/*` และ `bookings/*` พร้อมกัน หากที่นั่งใดไม่ใช่ Available แล้ว Rules จะปฏิเสธทั้งชุด ไม่เกิดการจองบางส่วน

4. โครงสร้างหน้าจอ Route และ Component

Route	หน้า	ต้อง login
/	HomePage	ไม่

/concerts, /search?q=	ConcertListPage	ไม่
/concerts/:id	ConcertDetailPage	ไม่ (กดหัวใจต้อง login)
/booking/:id	SeatBookingPage	ใช่
/favorites	FavoritesPage	ใช่
/my-tickets	ETicketPage	ใช่
/login	LoginPage (login/สมัคร)	ไม่
*	NotFoundPage	ไม่

```
App → AuthProvider → AppRoutes → Layout (Navbar, Outlet, Footer)
  pages: HomePage · ConcertListPage · ConcertDetailPage · SeatBookingPage
         FavoritesPage · ETicketPage · LoginPage · NotFoundPage
  common:   Status (loading/error/empty) · ConcertCard · CategoryCard
  features: SearchBar · SeatGrid → SeatItem · BookingSummaryBar
            FavoriteButton · TicketCard → QRCodeDisplay
  services: firebase.js · concertService.js    hooks: useAsync    utils: errors.js
```

ทุกหน้าที่โหลดข้อมูลใช้ `useAsync` + `Status` เพื่อแสดงสถานะกำลังโหลด / ผิดพลาด (พร้อมปุ่มลองใหม่) / ไม่พบข้อมูล UI เป็น responsive ด้วย CSS breakpoint 800px และ 1024px (เมนูเลื่อนแนวนอนบนมือถือ ตารางที่นั่ง 6 คอลัมน์)

5. วิธีติดตั้งและใช้งานระบบ

1. สร้าง Firebase Project เปิด Realtime Database แล้ว Import `firebase/firebase_data.json`
2. วาง Rules จาก `firebase/database.rules.json` แล้ว Publish; เปิด Authentication แบบ Email/Password
3. เพิ่ม Web app แล้วคัดลอก config ลง `.env` (ดู `.env.example`)
4. `npm install` → `npm run dev` (พัฒนา) / `npm run build` (Production Build)
5. Deploy: `firebase deploy --only hosting,database`

รายละเอียดครบถ้วนอยู่ใน `README.md`

6. ผลการทดสอบ

สถานะ: ตารางด้านล่างคือกรณีทดสอบที่ต้องรันกับระบบที่ deploy แล้ว (ฉบับเดียวกับ `docs/TEST_PLAN.md`) ผู้จัดทำต้องกรอกผลจริงและแนบภาพหน้าจอก่อนส่ง รายงานนี้ไม่ได้ระบุผลสำเร็จแทนการทดสอบจริง

ID	กรณีทดสอบ	ผลที่คาดหวัง	ผล
T01	เปิดหน้าแรก	loading แล้วแสดง 5 หมวด + 6 คอนเสิร์ต	☐
T02	ออฟไลน์แล้วเปิดหน้าแรก	ข้อความผิดพลาด + ปุ่มลองใหม่	☐
T03–T05	ค้นหามีผล / ไม่มีผล / กรองหมวด	รายการและข้อความถูกต้อง	☐
T06–T08	รายละเอียด / id ไม่มีจริง / route ไม่มีจริง	แสดงข้อมูล / ไม่พบคอนเสิร์ต / 404	☐
T09–T10	เข้าหน้าที่ต้อง login ขณะยังไม่ login	ไป /login แล้วกลับหน้าเดิมหลัง login	☐
T11–T15	สมัคร/เข้าสู่ระบบ ข้อมูลถูก-ผิด	ข้อความ validate ถูกต้อง; สมัครสำเร็จมี users/{uid}	☐
T16	เพิ่ม/ลบรายการโปรด	ปรากฏ/หายใน /favorites	☐
T17–T20	เลือกที่นั่ง จำกัด 6 ที่ ที่นั่ง Booked ยืนยันจอง	ยอดรวมถูก, กดที่ Booked ไม่ได้, ได้ E-Ticket	☐
T21	สองผู้ใช้จองที่นั่งเดียวกัน	คนหลังถูกปฏิเสธ ไม่จองซ้ำ	☐
T22–T24	สิทธิ์ข้อมูล (ตั๋วของคนอื่น / เขียน concerts / อ่าน users คนอื่น)	ถูกปฏิเสธ	☐
T25	Responsive 360 / 768 / 1280 px	ไม่มี scroll แนวนอน ใช้งานได้	☐
T26–T27	รีเฟรช route หลัง deploy / ตรวจ repo ไม่มี .env	ไม่ 404 / ไม่มีไฟล์ลับ	☐

บันทึกข้อผิดพลาดที่พบและการแก้ไข

#	ปัญหา	การแก้ไข	Commit

7. URL ของเว็บไซต์และ GitHub Repository

เว็บไซต์ที่ Deploy	_____ (กรอกหลัง deploy)
GitHub Repository	_____ (กรอกหลังสร้าง repo)
วิดีโอสาธิตระบบ	_____

เทคโนโลยี: React 18, Vite 5, React Router 6, Firebase 10 (Realtime Database, Authentication, Hosting)